Forecasting a Time Series: Chronological Splits, Windows, Recurrent Networks (Lab 6)

CS 53744 Machine Learning Project

Due Date : 11:59 PM, October 28, 2026

Instructor: Prof. Jongmin Lee

How to use this note:

- Read it before the lab session. The lab will not repeat this material.
- Covered: chronological splits and the embargo, building windows, the persistence baseline and scale-appropriate metrics, and the last hidden state of a recurrent network.
- Not covered: code and function calls. See the docstrings in the skeleton `src/lab06.py`.
- Rules for this lab (environment, tasks, submission) are in the `README.md` in this folder. Course-wide policy (grading and AI use) was given in the policy slides of the first class and is not repeated here.

1 Learning objectives

1. Explain why a random split leaks on a time series, and what a chronological split and an embargo each protect against. (Task 2)
2. Define exactly how a window and its target sit in time, and know what happens when they are off by one. (Task 1)
3. Explain why the standardisation constants and the metric's reference value come from the training segment only. (Task 4)
4. Build the persistence baseline and say when to report MAE, RMSE, or MASE. (Task 4)
5. Explain that a recurrent network reuses one set of weights, and pick the hidden state that goes to the head. (Task 3)

2 Background: the forecaster that says "same as yesterday"

A cafe manager wants to know how many cups will sell tomorrow, because that decides how much coffee to grind and how many staff to call in. You have three years of daily sales, one row per day.

There is a table and a column to predict, so the tempting move is the one from Lab 2: shuffle the rows and split. That scores beautifully, and the score is fake. A random shuffle puts one day in the test set while the day before and the day after stay in training. The model predicting that day has already seen the next one. It is sitting an exam with a view of the neighbouring desk.

Switch the model on for real and there is no neighbouring desk. At the moment you predict tomorrow, the day after has not happened, and it never will in time. The

assumption that has sat quietly under every lab since Lab 1, that rows have nothing to do with each other, breaks here.

So a time series is cut by the clock: learn from the earlier part, test on the later part. Then the second problem appears. A forecaster that learns nothing and simply answers "the same as yesterday" does rather well, because sales do not leap overnight. Your network only means something once it beats that lazy forecaster. Finding where the room to beat it is, and how much of it there is, is what this lab measures.

The four steps of Lab 6:

- **Windows:** turn one-day rows into (14 input days, 1 target day) pairs. The target must never sit inside the input. (Task 1)
- **Chronological split:** cut train, then validation, then test, from the front. No shuffling. Seven days are dropped between segments. (Task 2)
- **Model:** a GRU reads the window, and a linear head sits on the last hidden state. (Task 3)
- **Controlled comparison:** persistence against the network on the same windows, the same targets, and the same metrics. (Task 4)

3 Key concepts

3.1 What a time series breaks

Every split since Lab 2 has rested on rows being independent. On a time series today's sales are tightly tied to yesterday's, so that assumption does not hold.

Why a random split leaks:

- The days immediately before and after a test day land in training, so the model reads the answer off its neighbours.
- That information does not exist when the model is used. Like the scaler leak in Lab 2 it raises no error, only the score.

The chronological split:

- The earlier stretch is training, the middle is validation, the end is test, so no training date comes after a validation or test date (Figure 1).
- An *embargo* throws away a few points between segments. Lab 6 drops seven.

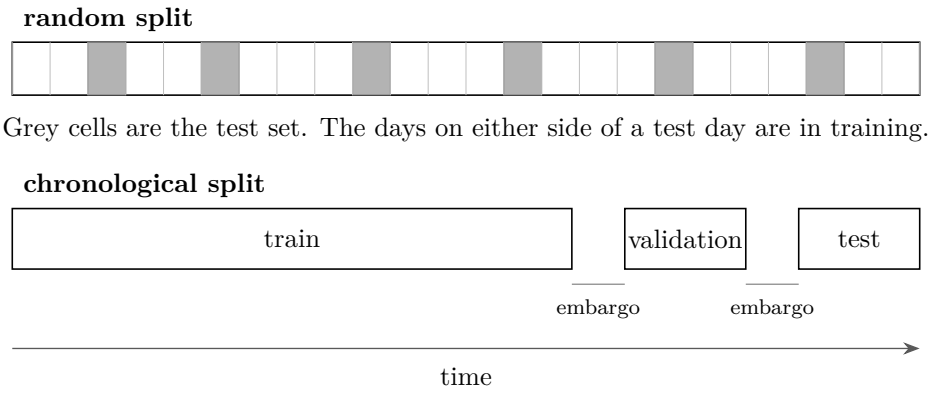


Figure 1. A random split and a chronological split. In the top row the neighbours of a test day sit in training, so the answer leaks out. In the bottom row the three segments run in time order with an embargo between them.

Spec (Task 2)

`n_test = int(round(test_frac * n))`
`n_val = int(round(val_frac * n))`
`n_train = n - n_val - n_test - 2*gap`
Train is `0 .. n_train-1`, validation is `n_train+gap .. n_train+gap+n_val-1`, and test is `n_train+2*gap+n_val .. n-1`.

Assumptions and limits:

- One chronological split is one measurement. In practice you move the cut point and measure several times (*walk-forward validation*), but Lab 6 cuts once.
- With windows built inside each segment and a horizon of 1, there is no leak even at `gap=0`. The seven days go because the moment you add a feature such as a moving average, values from both sides mix at the boundary.

3.2 Windows: the target sits outside the input

A *recurrent network* takes a sequence of values, so the series is cut into (input window, target) pairs: `window` consecutive days, and a single target point a *horizon* of steps after the window ends (Figure 2).

Spec (Task 1)

`X[i] = s[i : i+window]`, `y[i] = s[i + window + horizon - 1]`
There are `N - window - horizon + 1` pairs. With `horizon=1` the target is the point immediately after the window. Every value in `X[i]` must come strictly before `y[i]` in time.

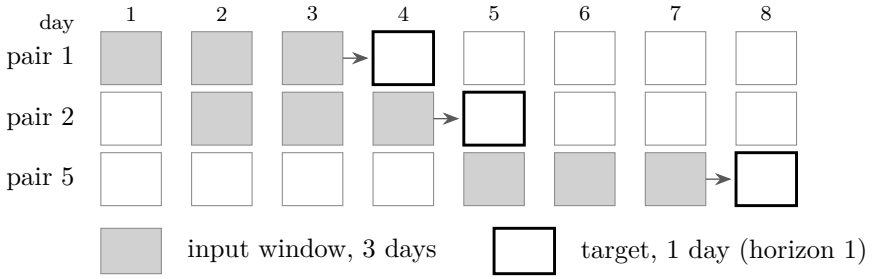


Figure 2. The pairs built from eight points. The window is three consecutive days and the target is the next one. The window slides one day at a time, giving five pairs, and the target cell is always to the right of the grey cells.

What an off-by-one does:

- Make the target the last value of the window and the answer is inside the input (§4 shows what that does to the numbers).
- Push it one step the other way and you have built a model that predicts two days ahead. The error grows and no exception is raised.

3.3 The scaler and the reference value come from training only

Lab 2 called it leakage to fit the scaler on the full dataset. On a time series the metric's reference value belongs in the same category.

Computed on the training segment only:

- **Standardisation mean and standard deviation:** computed over the whole series, the level of the test segment enters the transform. A hidden test reads `scale_mean` back and compares it with the training mean.
- **The MASE reference value:** computed over the whole series, a test segment that moves a lot enlarges the denominator and improves the score for free.
- **Windows and targets:** built inside each segment, or you get windows that cross a boundary.

Also worth knowing:

- Predictions come out in standardised units, so convert them back to cups before computing any metric. Skip that and the error comes out in standard deviations, a number that means nothing in cups. The standard deviation is the population one (`ddof=0`).
- The validation segment is used only to choose the weights from the best epoch.

3.4 Recurrent networks reuse one set of weights

A recurrent network reads the values in the window one at a time and updates a *hidden state* as it goes. The hidden state summarises everything read so far, and the weights that update it are the same at every step (Figure 3).

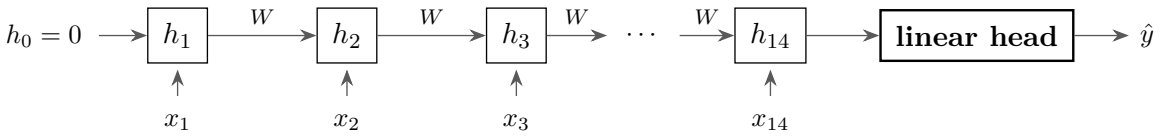


Figure 3. One window, unrolled. The network reads one value per step and updates the hidden state. The same W is used at every step, and the only state that reaches the head is h_{14}, after the whole window has been read.

What sharing the weights buys:

- The parameter count is the same whether the window is 14 or 30, and a rule like "higher than three days ago" is learned once and applies at every position in the window.
- Repeating the same multiplication makes distant gradients vanish or explode. That is why Lab 6 uses a GRU, whose gates control the size of each update.

Which hidden state to use:

- Use the hidden state at the **last time step**, after the whole window has been read. With `batch_first=True`, `nn.GRU` returns `(batch, time, hidden)`, so the last step is the end of the middle axis.
- Slicing the end of the first axis leaves the last sample of the batch instead. If the batch size equals the window length the shape check still passes, so a completely different value goes through with no error. A hidden test uses batch 4 and 9 time steps for this.

3.5 The persistence baseline and scale-appropriate metrics

The *persistence forecast* answers with the most recent observation. It learns nothing, and the last cell of the window already holds the number. It plays the role the majority-class baseline played in Lab 2, it is far stronger, and its error is measured three ways (Table 1).

Spec (Task 4)

```
mae = mean(|y_true - y_pred|)
rmse = sqrt(mean((y_true - y_pred)^2))
mase = mae / naive_scale
```

`naive_scale` is the mean absolute one-step change of the training series. All three are computed in cups and rounded to 4 decimals.

Table 1. The three metrics. The first two are in the units of the series and MASE has none. MASE is the only one that still means something next to another cafe. In the lab all three are measured on the test segment.

Metric	Definition	Unit	How to read it
MAE	mean absolute error	cups	how many cups you are off by
RMSE	root mean squared error	cups	large errors weigh more
MASE	MAE over the mean one-step change of the training series	none	below 1 is smaller than the day-to-day movement

How to read it:

- MAE and RMSE are in cups, so they mean something only next to the size of this cafe. An error of 10 cups is small for a shop that sells 1,000 a day and large for one that sells 50.
- MASE divides by how much the training series moves from one day to the next. A MASE of 0.5 means the error is half of that movement, and a reader who does not know the cafe can still read it.
- Persistence need not land at exactly 1 on the test segment. The denominator comes from the training segment, so a test segment that moves more or less than training pushes it away from 1. Compare two forecasters by their MASE on the same test windows.

Assumptions and limits:

- Persistence assumes the value changes slowly. It breaks wherever the series jumps from one day to the next. On strongly seasonal data the seasonal persistence forecast, repeating the value from a week ago, is stronger.
- A 14-day window cannot see an annual cycle, because the model never gets to look at one.

4 Worked example: windows and errors from eight days

Work through the eight days in Table 2, with a window of 3 and a horizon of 1.

Table 2. The eight days of the worked example. Monday through the following Monday. The value drops sharply over the weekend and climbs back on Monday.

Day	1	2	3	4	5	6	7	8
Weekday	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon
Cups	180	190	186	182	170	140	138	188

Building the pairs:

1. The number of pairs is $8 - 3 - 1 + 1 = \mathbf{5}$.

2. The first pair takes days 1 to 3 and targets day 4. The window slides one day at a time until the last pair takes days 5 to 7 and targets day 8 (Table 3).

Table 3. The five pairs and the errors of two forecasters. Persistence answers with the last value of the window, the 3-day mean with the average of all three. The two right-hand columns are the absolute error on each pair.

Pair	Input window	Target	Persist.	3-day mean	Persist. err.	Mean err.
1	180, 190, 186	182	186	185.33	4	3.33
2	190, 186, 182	170	182	186.00	12	16.00
3	186, 182, 170	140	170	179.33	30	39.33
4	182, 170, 140	138	140	164.00	2	26.00
5	170, 140, 138	188	138	149.33	50	38.67

Computing the metrics:

1. Reference value: the absolute day-to-day changes are 10, 4, 4, 12, 30, 2, 50, which sum to 112. There are seven changes, so `naive_scale` $= 112/7 =$ **16.0**.
2. Persistence MAE $= (4 + 12 + 30 + 2 + 50)/5 = 98/5 =$ **19.6**
3. 3-day mean MAE $= (3.33 + 16.00 + 39.33 + 26.00 + 38.67)/5 =$ **24.6667**
4. Persistence MASE $= 19.6/16.0 =$ **1.225**, 3-day mean MASE $= 24.6667/16.0 =$ **1.5417**

What to take from it:

- The 3-day mean looks more sophisticated and is worse, because an average always follows a turn one beat late.
- Both forecasters are far off on days 6 and 8, where the weekend starts and ends. That is where a model with access to the weekday can win.
- Persistence goes above 1 here because the denominator also averages the first two changes (10 and 4), which no target uses, while the numerator sees only the last five.
- Make the target the last value of the window by mistake and you get six pairs and a persistence MAE of **exactly 0**. **A baseline error near zero means the window definition is wrong.**

5 Common mistakes

1. **Shuffling the rows.** Calling `train_test_split` puts the neighbours of a test day into training, and the error drops sharply.
2. **The window overlapping the target by one.** As in §4, the baseline error falls to 0.
3. **Standardising on the whole series.** The level of the test segment enters the transform.
4. **Slicing the hidden state on the batch axis.** There is no shape error, so the wrong value passes the check. Take the end of the middle axis.
5. **Not fixing the seed.** Call `set_seed()` at the top of `run_experiment`.

6 How this maps to the lab tasks

Table 4 matches each task to the section it draws on.

Table 4. Tasks matched to concept sections. Worth checking once before you write the report.

Lab task	Concept	What grading looks at
Task 1. Windows	§3.2, §4	the target outside the window, the right number of pairs
Task 2. Chronological split	§3.1	no shuffling, order and embargo as specified
Task 3. Recurrent model	§3.4	structure, the hidden state of the last time step
Task 4. Experiment	§3.3, §3.5	scaling from training only, a controlled comparison, reproducibility
Report figures 1–3	§3.1, §3.5	the split, where the forecasts fail, scale-appropriate metrics

7 Self-check

1. You split the series at random and the error halved. Why is that not an achievement?
2. With a series of length 100, a window of 10, and a horizon of 3, how many pairs do you get?
3. Persistence comes back with MASE 1.4 on the test segment. Model or data?
4. If the window grows from 14 to 60 days, what happens to the parameter count?
5. If you took the hidden state of the first time step, what is the model predicting?

These are fair game for the oral check. You must be able to explain everything you submit.

8 Further reading

- Hyndman & Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed., §5.8, §5.10. `https://otexts.com/fpp3/`
- Hyndman & Koehler, *Another look at measures of forecast accuracy*, 2006. `doi: 10.1016/j.ijforecast.2006.03.001`